

Guess Market — Exercise 2 (JavaFX)

README, System Overview, Design Choices & Assumptions

★ BONUS FEATURES IMPLEMENTED ★

1. **Skin switching (Bonus 1).** Six looks — a plain *Default* plus *Light*, *Dark*, *Solarized*, *Nord*, *Sepia*. Each skin changes the whole-window background, the button appearance, and the **font family and size of every label**. Chosen from **System** → **Theme**.
2. **Operation animations (Bonus 2).** Two ≤ 2 s animations: the *Events* detail panel fades + slides in when an event is selected, and the *Users* detail panel does the same when a user is selected. Both can be turned off from **System** → **Animations** (a Save button applies the change). Note: the animations ship **enabled** so they are visible immediately; untick either box and Save to disable.
3. **Graphs (Bonus 3).** In an event's details, a price-over-time line chart per option; in a user's details, a balance-over-time line chart.
4. **Create a new event (Bonus 4).** **File** → **Add Event...** opens a form (market maker, trading method, then the method-specific fields). The chosen participant becomes the event's market maker and the event then behaves like any other.

1. Submission & Contact Information

Full Name	ID Number	Contact Email
Alon Golembo	207684085	alon.golembo@gmail.com
Snir Bar Shimon	315370106	snir221195@gmail.com

The exercise is uploaded once, under **Alon Golembo**'s account.

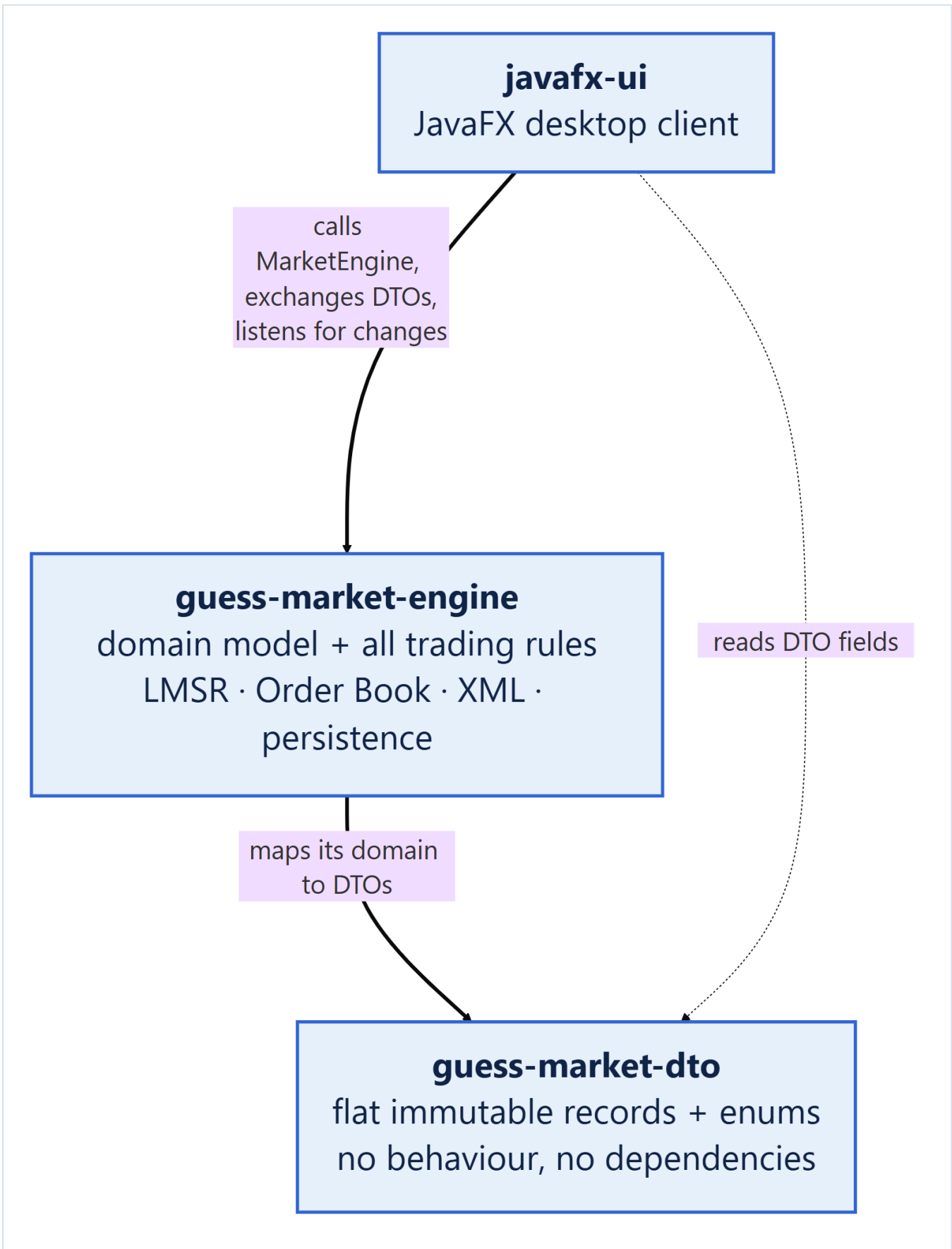
2. GitHub Repository

<https://github.com/alongolembo/GuessMarket> — full source, commit history, module declarations and the engine test suite.

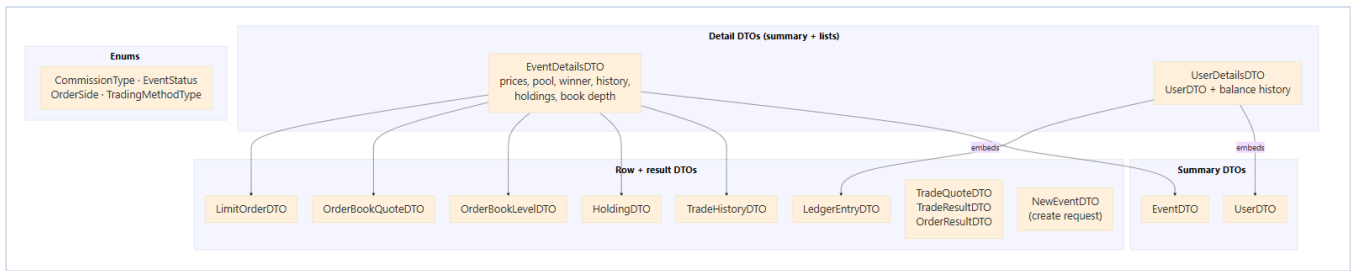
3. System Overview

Three modules with a strict one-way dependency `javafx-ui` → `guess-market-engine` → `guess-market-dto`, enforced by `module-info`. The UI never touches a domain object: it calls the `MarketEngine` interface with flat immutable DTOs, renders the DTOs it gets back, and re-reads everything whenever the engine publishes a "data changed" event.

3.1 Modules

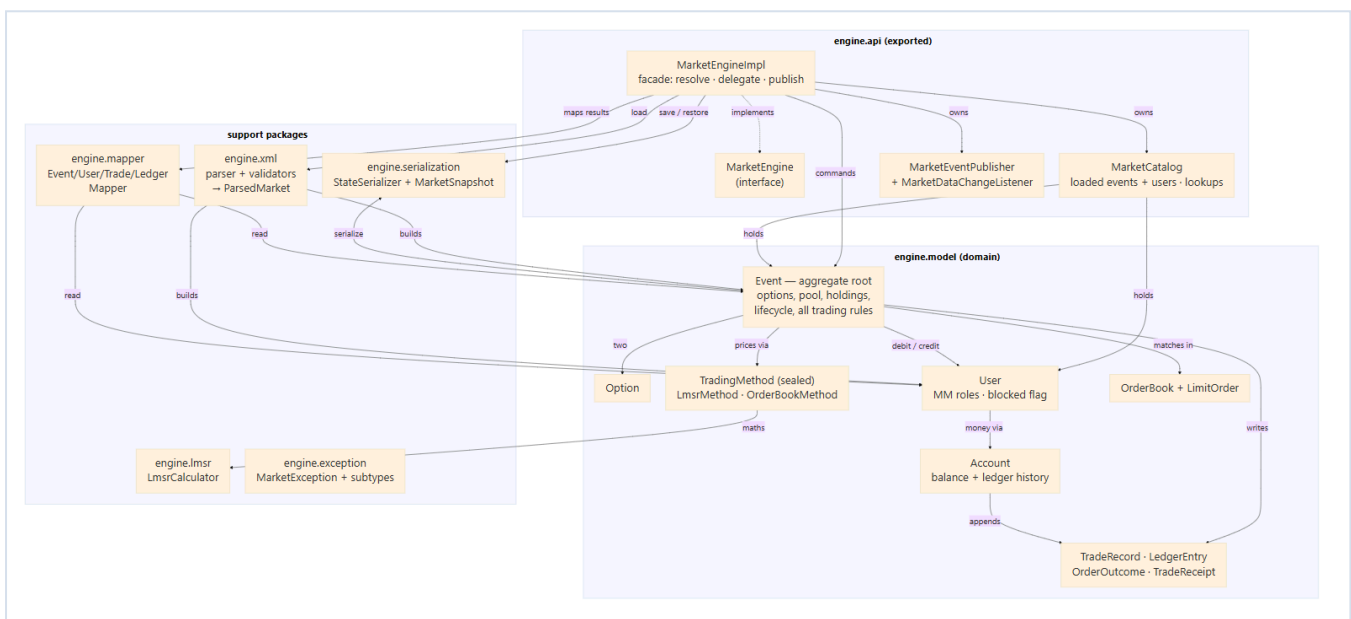


3.2 guess-market-dto



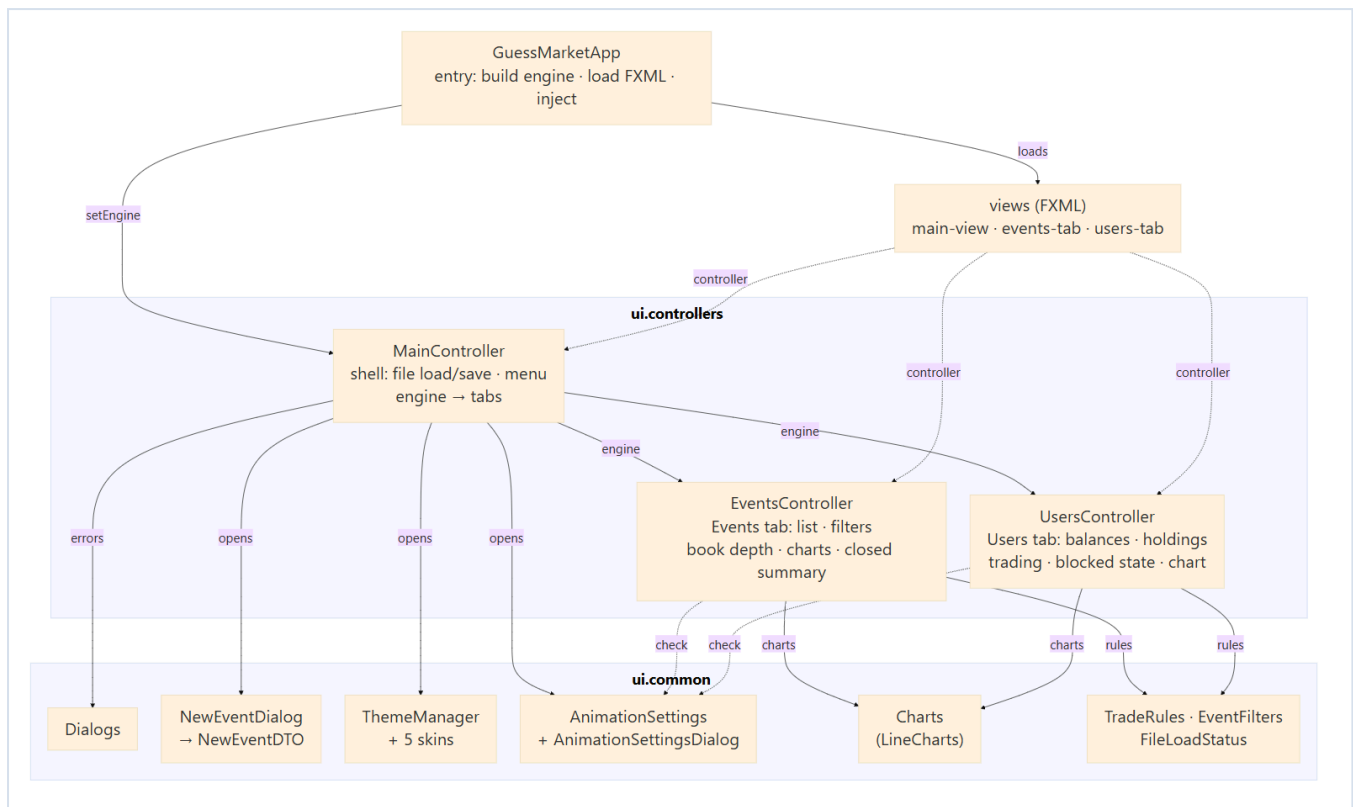
Every DTO is a flat, immutable `record`. A *summary* DTO (`EventDTO` , `UserDTO`) is what tables show and what the UI passes back as a command argument; a *detail* DTO embeds its summary DTO plus the heavy lists. `EventDTO` and `UserDTO` never reference each other — they cross-link by id / name.

3.3 guess-market-engine



`MarketEngineImpl` owns no rules: it resolves the DTO to a domain object via `MarketCatalog`, tells the **Event aggregate root** to do the work (every trading and lifecycle rule lives there), runs the result through a `*Mapper` and fires `MarketEventPublisher`. `TradingMethod` is the single polymorphic seam for LMSR-vs-Order-Book behaviour (callers never switch on its type). Money moves only through `Account`, which also keeps the append-only balance-over-time ledger. XML load and Java-serialization restore are the only two ways a whole market graph enters the catalog.

3.4 javafx-ui



`GuessMarketApp` boots one `MarketEngineImpl` and the main window; `MainController` injects that engine into the two tab controllers, which register as `MarketDataChangeListener`s. All bonuses live in `ui.common`.

3.5 Main classes (new since Exercise 1)

Module	Class	Responsibility
javafx-ui	GuessMarketApp	Application entry point. Builds the engine, loads <code>main-view.fxml</code> , injects the engine into the controller.
javafx-ui	MainController	Window shell: the menu bar, the XML file-chooser + background load Task + progress bar, Save/Load state, the Add-Event / Theme / Animations dialogs, and propagating the engine to the tab controllers.
javafx-ui	EventsController	Events tab: the filterable events table (method / status / commission), the per-event drill-in (LMSR details, or the two order books with Last/Bid/Ask/Mid/Spread), participant holdings, the closed-event winner + total-shares summary, and the price-over-time charts.
javafx-ui	UsersController	Users tab: the users table, the selected user's balance / blocked state / participating events, the per-event involvement summary (shares per option, and for a closed event the winner and the user's net result), LMSR buying, Order-Book order entry & open-order cancellation, and the balance-over-time chart.
javafx-ui	ThemeManager	Bonus 1. Loads the five skin stylesheets and swaps them on the <code>Scene</code> ; a "Default" entry clears the stylesheet.
javafx-ui	AnimationSettings / AnimationSettingsDialog	Bonus 2. In-memory on/off switches for the two animations, and the Save dialog that writes them.
javafx-ui	Charts	Bonus 3. Builds the JavaFX <code>LineChart</code> s (option price history, user balance history) from detail DTOs.
javafx-ui	NewEventDialog	Bonus 4. The modal add-event form; validates input and returns a <code>NewEventDTO</code> .
javafx-ui	Dialogs / TradeRules / EventFilters	Shared helpers: uniform error/info/confirm boxes; pure (unit-tested) decisions about which actions a (user, event) pair allows; the events-table filter predicate.
guess-market-engine	MarketEngine / MarketEngineImpl	The only entry to the engine. Resolves DTOs to domain objects, delegates, publishes a change event. New in Ex 2: <code>getAllUsers</code> , <code>getUserDetails</code> , <code>placeOrder</code> , <code>cancelOrder</code> , <code>createEvent</code> , <code>save/loadState</code> .
guess-market-engine	MarketCatalog	Holds the currently loaded events and users; does every id / name lookup with a clear "load a file first" message.
guess-market-engine	MarketEventPublisher / MarketDataChangeListener	Fan-out of "market data changed" so the UI refreshes after every mutation.

guess-market-engine	Event	The aggregate root. Owns its two options, cash pool, participants, holdings, trade history and lifecycle (NOT_ACTIVE → ACTIVE → CLOSED), and enforces every rule: who may open/close, subsidy / initial-allocation funding, LMSR buys, Order-Book matching & minting, settlement payouts, and the blocked-user rule.
guess-market-engine	User	A participant: name, an Account , the events they market-make, and a blocked flag set when their balance is forced negative.
guess-market-engine	Account	A cash balance. The single choke point for money — debit / credit (a normal overdraw is refused) plus debitAllowingOverdraw for the blocked-user path. Every move appends a LedgerEntry .
guess-market-engine	TradingMethod (sealed) → LmsrMethod, OrderBookMethod	Method-specific behaviour: pricing, initial subsidy / allocation, trade cost, config validation. Callers ask the method; they never switch on its concrete type.
guess-market-engine	OrderBook / LimitOrder	Per-option bid and ask books kept in strict price-then-time priority; the matching, minting and no-escrow fill-time clamping are driven by Event .
guess-market-engine	LedgerEntry / TradeRecord	Immutable audit records — one per balance move (with the resulting balance, for the chart) and one per trade.
guess-market-engine	engine.xml (GuessMarketXmlParser, XmlMarketReader, MarketAssembler, EventXmlValidator, UserXmlValidator, MarketMakerValidator)	Reads the market XML with JAXB and validates it: unique event ids, exactly two non-empty options, commission 0–90, a single valid trading method, unique user names, initial cash > 0, market-maker references point at real events, exactly one market maker per event.
guess-market-engine	engine.mapper (EventManager, UserMapper, TradeMapper, LedgerMapper)	Convert domain objects to DTOs — one direction only, no recursion.
guess-market-engine	engine.serialization (StateSerializer, MarketSnapshot)	Save / restore the whole (events, users) object graph via Java serialization (the Exercise-1 bonus, still working).
guess-market-engine	LmsrCalculator	The LMSR maths: cost $C(q_0, q_1) = b \cdot \ln(e^{(q_0/b)} + e^{(q_1/b)})$, implied price, trade cost, and the initial subsidy $C(0, 0) = b \cdot \ln 2$. Numerically stabilised with a max-shift.
guess-market-dto	com.guessmarket.dto.*	All the cross-boundary records (EventDTO , EventDetailsDTO , UserDTO , UserDetailsDTO , NewEventDTO , TradeHistoryDTO , LedgerEntryDTO , order-book DTOs, result DTOs) and enums. No behaviour, no dependencies.

4. Key Design Choices & Assumptions

- **Layering.** Three modules with `module-info` declarations; the UI sees only `engine.api` + the DTOs. The engine's `model`, `lmsr`, `xml`, `mapper`, `serialization` packages are not exported.
- **Immutable DTOs.** The engine never leaks a mutable entity; the UI can't accidentally mutate market state.
- **Event is the aggregate root.** No plain setters; every rule about options, money, holdings and lifecycle is enforced inside it. `MarketEngineImpl` is a thin facade.
- **Money is confined to Account** (as `double` dollars). Every debit/credit appends a `LedgerEntry` that records the resulting balance, which is what the balance-over-time chart reads directly.
- **Order Book has no escrow.** Cash and shares move only when an order actually fills. A placed order is checked against the placer's *current* balance / free shares (a bid needs the full cost incl. on-purchase commission; an ask needs the shares net of the user's own other asks). Price is capped at `d - 0.01`.
- **Blocked users (negative-balance rule).** A user cannot normally go negative. But a resting bid can become unhonourable if its owner has since spent that money on another order. When that fill happens we *do* honour it: the owner's balance is forced negative, they are **blocked** (the engine refuses every subsequent action of theirs), a dialog notifies the checker, and a red banner marks the user. There is no way to top the account back up (per the spec). If instead the *seller* of a resting ask can no longer deliver the shares, that order is simply dropped.
- **Filters.** The three event filters are combo boxes, each with an explicit "All" option selected by default (a pragmatic reading of the "toggle buttons" hint — one control, clearly shows the active choice).
- **Loading.** File-chooser dialog only. Parsing runs on a JavaFX `Task` with a progress bar; since parsing is near-instant, a ~1.3 s paced delay is added so the indicator is visible. All the Exercise-1 input validations still apply, plus the Exercise-2 user checks.
- **Closed events.** The event drill-in shows the winning option and total shares bought per option. A user's per-event view shows their shares per option and, once closed, the winner and their net cash result on that event (summed from the ledger).
- **Binary events.** Exactly two options per event, as in the schema.
- **Created events** get the next free integer id (max existing + 1). Before creation the engine checks the chosen market maker can afford the trading-method subsidy; if not, an error dialog explains why. A created event starts `NOT_ACTIVE` — the creator activates it like any other.
- **Resize.** The window is resizable; content sits in scroll panes and split panes and a minimum window size stops controls being clipped.
- **State save/load** (Exercise-1 bonus) writes/reads the full object graph with Java serialization and is offered from the **File** menu, separate from XML loading.

5. Tests

The engine module carries a JUnit 5 suite (run from IntelliJ) covering the LMSR maths, XML validation, the full Order-Book lifecycle (matching, minting, market buys, settlement), state round-trips, the balance ledger, event creation, and the blocked-user path. The UI is exercised manually against the course test files.